1. Introduction. In the fall of 1972, Bob Floyd posed the following "toy problem" to the first-year students in Stanford's Ph.D. program, and Donald E. Knuth later discussed it in his essay *Are Toy Problems Useful?* (reprinted in *Selected Papers on Computer Science*, 1996):

> "The numbers $\sqrt{1}, \sqrt{2}, \ldots, \sqrt{50}$ are to be partitioned into two parts whose sums are nearly equal; find the best such partition you can, using less than 10 seconds of computer time."

Since

$$(\sqrt{1} + \sqrt{2} + \cdots + \sqrt{50})/2 = 119.51790\,03017\,60392\,24702\ldots,$$

each part should come as close as possible to that half-sum; equivalently we seek a subset of $\{\sqrt{1}, \ldots, \sqrt{50}\}$ whose sum least exceeds it.

A brute-force search is hopeless. There are $2^{50} \approx 10^{15}$ subsets, and the ones near the middle are packed densely: more than 10^{14} of them have sums within an interval of length less than 70 around the target, so the best partition should beat the target by something like 10^{-13} or so. The interesting part is finding that needle without examining 10^{15} subsets.

This program does so in under two seconds. It combines four classical ideas, each of which Knuth's essay singles out as the kind of technique a good toy problem teaches: separating the *perfect squares* as an integer adjustment, a *meet-in-the-middle* split with a sorted lookup table, *Gray-code* subset enumeration, and *compensated summation* backed by an arbitrary-precision check. The target value, to full **float64** precision, is the one constant the whole search aims at.

```
package main
import (
    "flag"
    "fmt"
    "math"
    "math/big"
    "math/bits"
    "sort"
    "time"
)
const target = 119.51790030176039224702 // (√1 + √2 + ... + √50)/2
⟨ Subroutines 7 ⟩
func main() {
    ⟨ Parse the command-line flags 2 ⟩
    ⟨ Set the perfect squares aside 3 ⟩
    ⟨ Build the sorted table of A-subset sums 6 ⟩
    ⟨ Set up the search state 9 ⟩
    ⟨ Define the eval helper 10 ⟩
    ⟨ Define the probe helper 11 ⟩
    ⟨ Scan B with a Gray code, probing each subset 12 ⟩
    ⟨ Reconstruct the winning partition 13 ⟩
    ⟨ Verify in high precision and report 14 ⟩
}
```

2. The `-v` flag turns on timing lines and the *math/big* verification; without it the program prints just the two groups. We start the clock right away so the reported time covers the whole search.

```
⟨ Parse the command-line flags 2 ⟩ ≡
    verbose := flag.Bool("v", false, "verbose output")
    flag.Parse()
    start := time.Now()
```

This code is used in section 1.

3. An Approach to Floyd's Problem. Among $1, \dots, 50$ exactly seven are perfect squares: 1, 4, 9, 16, 25, 36 and 49. Their roots $1, 2, \dots, 7$ are *integers*, so moving a square between the two groups changes a group's sum by a whole number. Moreover every integer from 0 to 28 can be written as a sum of a subset of $\{1, 2, \dots, 7\}$, so the squares give us a free adjustment knob $k \in [0, 28]$ to be applied at the very end.

This is the key simplification: during the hard part of the search we may ignore the integer part of a sum entirely and worry only about its *fractional* part, knowing that the squares can later fix the integer part up to 119. That leaves the 43 non-square numbers, whose roots are irrational, as the only ones that need real searching.

⟨ Set the perfect squares aside 3 ⟩ ≡

```
squares := map[int]bool{1: true, 4: true, 9: true, 16: true, 25: true, 36: true, 49: true}
var nonSquares []int
for k := 1; k ≤ 50; k++ {
    if !squares[k] {
        nonSquares = append(nonSquares, k)
    }
}
Aitems := nonSquares[:21]
Bitems := nonSquares[21:]
```

This code is used in section 1.

4. Even 2^{43} subsets of the non-squares is far too many to enumerate. The standard remedy is to *meet in the middle*: split the 43 numbers into two halves A (the first 21) and B (the last 22), and trade exponential time for exponential space on one side. We precompute and *sort* all $2^{21} \approx 2$ million subset sums of A; then, for each of the $2^{22} \approx 4$ million subset sums of B, a single binary search finds the A-subset that best completes it. The total work is on the order of $2^{22} \log 2^{21}$ operations instead of 2^{50} — comfortably inside Floyd's budget.

Each table entry remembers three things: the fractional part of the subset sum (the key we search on), the full sum (needed to pick the integer knob later), and a bit *mask* recording which of the 21 numbers are in the subset.

```
type entry struct {
    frac float64
    sum float64
    mask uint32
}
```

5. We sort the table by fractional part. A tiny *sort.Interface* does the job.

```
type byFrac []entry
func (s byFrac) Len() int { return len(s) }
func (s byFrac) Less(i, j int) bool { return s[i].frac < s[j].frac }
func (s byFrac) Swap(i, j int) { s[i], s[j] = s[j], s[i] }
```

6. Building the table calls *genAll* (developed in the next two sections) and sorts the result. For the binary search we also pull the sorted fractional parts into a plain []**float64**, which *sort.SearchFloat64s* can probe directly. We record the target's own fractional part once, since every probe compares against it.

⟨ Build the sorted table of A-subset sums 6 ⟩ ≡

```
tA := time.Now()
Aents := genAll(Aitems)
sort.Sort(byFrac(Aents))
Afracs := make([]float64, len(Aents))
for i, e := range Aents {
    Afracs[i] = e.frac
}
if *verbose {
    fmt.Printf("A (2^%d) generate+sort: %v\n", len(Aitems), time.Since(tA))
}
targetFrac := target - math.Floor(target)
nA := len(Aents)
```

This code is used in section 1.

7. We are about to add up millions of square roots, hunting for an answer that is correct in its *thirteenth* decimal place. Naive **float64** accumulation would drift well before then. The fix is *compensated summation*: alongside the running *sum* we carry a small correction term c that captures the low-order bits lost in each addition. This is the Neumaier (Kahan–Babuška) refinement, which handles the case where the new value is larger in magnitude than the running total.

⟨ Subroutines 7 ⟩ ≡

```
func kbn(sum, c, v float64) (float64, float64) {
    t := sum + v
    if math.Abs(sum) ≥ math.Abs(v) {
        c += (sum - t) + v
    } else {
        c += (v - t) + sum
    }
    return t, c
}
```

This code is also defined in section 8.

This code is used in section 1.

8. Enumerating subsets with a Gray code. To visit all 2^n subsets cheaply we walk them in *Gray-code* order, where consecutive subsets differ by exactly one element. Then each step adds or removes a single $\sqrt{k}$, updating the running sum in $O(1)$ instead of re-adding up to 22 terms. The element that flips at step i is the one in the position of the lowest set bit of i, which *bits.TrailingZeros32* gives us directly; whether it joins or leaves the subset depends on whether that bit is currently set in *mask*.

genAll applies this to a list of numbers and returns the full table of entries, each sum maintained with the compensated *kbn* step so the Gray-code updates stay accurate to the last bit or two. Index 0 (the empty subset, sum 0) is left as the zero **entry**.

⟨ Subroutines 7 ⟩ +≡

```
func genAll(items []int) []entry {
    n := len(items)
    size := uint32(1) << uint(n)
    vals := make([]float64, n)
    for i, k := range items {
        vals[i] = math.Sqrt(float64(k))
    }
    out := make([]entry, size)
    var sum, c float64
    var mask uint32
    for i := uint32(1); i < size; i++ {
        pos := uint(bits.TrailingZeros32(i))
        var v float64
        if mask&(1<<pos) != 0 {
            mask &^= 1 << pos
            v = -vals[pos]
        } else {
            mask |= 1 << pos
            v = vals[pos]
        }
        sum, c = kbn(sum, c, v)
        full := sum + c
        out[i].frac = full - math.Floor(full)
        out[i].sum = full
        out[i].mask = mask
    }
    return out
}
```

9. With the A table built and sorted, we stream the subset sums of B and, for each, ask the table for the best companion. The search state records the best pair found so far: the smallest leftover error *bestDiff*, the two sums and masks that achieved it, and the integer knob *bestK*.

The values of the B numbers, like A's, are taken as square roots once up front, so the Gray-code loop need only add and subtract them.

⟨ Set up the search state 9 ⟩ ≡

```
bestDiff := math.Inf(1)
var bestAsum, bestBsum float64
var bestAmask, bestBmask uint32
var bestK int
n := len(Bitems)
size := uint32(1) << uint(n)
Bvals := make([]float64, n)
for i, k := range Bitems {
    Bvals[i] = math.Sqrt(float64(k))
}
```

This code is used in section 1.

10. Given an index *ai* into the A table and a B-subset (its sum and mask), *eval* measures how good the combined partition is. The two real-valued sums contribute $Asum + Bsum$; the squares must supply the rest, $kf = target - total$, as an integer. We round kf to the nearest integer k, clamp it to the representable range $[0, 28]$, and take the leftover *diff* as the quality of the fit. Whenever a pair beats the incumbent we record it.

⟨ Define the *eval* helper 10 ⟩ ≡

```
eval := func(ai int, Bsum float64, Bmask uint32) {
    Asum := Aents[ai].sum
    total := Asum + Bsum
    kf := target - total
    k := min(max(int(math.Round(kf)), 0), 28)
    diff := math.Abs(kf - float64(k))
    if diff < bestDiff {
        bestDiff = diff
        bestAsum = Asum
        bestBsum = Bsum
        bestAmask = Aents[ai].mask
        bestBmask = Bmask
        bestK = k
    }
}
```

This code is used in section 1.

11. *probe* is where the meet-in-the-middle pays off. For a given B-subset we want an A-subset whose fractional part makes $Asum + Bsum$ land on the target modulo 1 (the integer part being the squares' job). The desired fractional part is

$$wantA = \big(\mathrm{frac}(target) - \mathrm{frac}(Bsum)\big) \bmod 1,$$

so we binary-search the sorted *Afracs* for it. Because fractional parts live on a circle — $0.999\ldots$ is adjacent to $0.000\ldots$ — the best entry is one of the *two circular neighbors* of the insertion point, so we evaluate both, wrapping from the end of the table to the front and vice versa.

⟨ Define the *probe* helper 11 ⟩ ≡

```
probe := func(Bsum float64, Bmask uint32) {
    fracB := Bsum - math.Floor(Bsum)
    wantA := targetFrac - fracB
    if wantA < 0 {
        wantA += 1
    }
    // checking the two circular neighbors is sufficient
    idx := sort.SearchFloat64s(Afracs, wantA)
    if idx < nA {
        eval(idx, Bsum, Bmask)
    } else {
        eval(0, Bsum, Bmask)
    }
    if idx > 0 {
        eval(idx-1, Bsum, Bmask)
    } else {
        eval(nA-1, Bsum, Bmask)
    }
}
```

This code is used in section 1.

12. Now we sweep all 2^{22} subsets of B in Gray-code order — the same single-bit-flip trick as *genAll*, but *streaming*: each subset sum is probed against the table and then discarded, so B is never stored. The empty subset is probed first, before the loop. Unlike Knuth's 1976 program, which had to sample only a fraction of B within the time limit, this exhaustive sweep finishes in well under the ten seconds, so it finds the true optimum rather than a lucky near-miss.

⟨ Scan B with a Gray code, probing each subset 12 ⟩ ≡

```
tB := time.Now()
probe(0, 0)
var sumB, cB float64
var maskB uint32
for i := uint32(1); i < size; i++ {
    pos := uint(bits.TrailingZeros32(i))
    var v float64
    if maskB&(1≪pos) ≠ 0 {
        maskB &^= 1 ≪ pos
        v = -Bvals[pos]
    } else {
        maskB |= 1 ≪ pos
        v = Bvals[pos]
    }
    sumB, cB = kbn(sumB, cB, v)
    probe(sumB+cB, maskB)
}
if *verbose {
    fmt.Printf("B (2^%d) scan+probe: %v\n", len(Bitems), time.Since(tB))
}
elapsed := time.Since(start)
```

This code is used in section 1.

13. Reconstructing the partition. The winning masks tell us which non-squares belong to the first group: bit i of *bestAmask* selects *Aitems*[i], and likewise for *bestBmask* over *Bitems*. Then the integer knob *bestK* is realized as actual squares. Decomposing *bestK* greedily into a subset of $\{7, 6, \ldots, 1\}$ always succeeds for any value in $[0, 28]$ (try the largest part that still fits, repeat), and each part i contributes the square i^2. Everything not placed in the first group forms the second.

⟨ Reconstruct the winning partition 13 ⟩ ≡

```
var group []int
for i, k := range Aitems {
    if bestAmask&(1≪uint(i)) ≠ 0 {
        group = append(group, k)
    }
}
for i, k := range Bitems {
    if bestBmask&(1≪uint(i)) ≠ 0 {
        group = append(group, k)
    }
}
rem := bestK
for i := 7; i ≥ 1; i-- {
    if rem ≥ i {
        group = append(group, i*i)
        rem -= i
    }
}
sort.Ints(group)
inGroup := map[int]bool{}
for _, k := range group {
    inGroup[k] = true
}
var other []int
for k := 1; k ≤ 50; k++ {
    if !inGroup[k] {
        other = append(other, k)
    }
}
```

This code is used in section 1.

14. The search ran entirely in **float64**, so its own report of the error cannot be trusted beyond a dozen digits. To state the result honestly we recompute both group sums from scratch with *math/big* at 200-bit precision (about 60 decimal digits) and print their difference. The plain-**float64** group sums are also computed, for the headline output.

⟨ Verify in high precision and report 14 ⟩ ≡

```
sumG, sumO := 0.0, 0.0
for _, k := range group {
    sumG += math.Sqrt(float64(k))
}
for _, k := range other {
    sumO += math.Sqrt(float64(k))
}
bigSum := func(ks []int) *big.Float {
    s := new(big.Float).SetPrec(200)
    for _, k := range ks {
        sq := new(big.Float).SetPrec(200).SetInt64(int64(k))
        sq.Sqrt(sq)
        s.Add(s, sq)
    }
    return s
}
bigG := bigSum(group)
bigO := bigSum(other)
bigDiff := new(big.Float).SetPrec(200).Sub(bigG, bigO)
```

This code is also defined in section 15.

This code is used in section 1.

15. Finally we print. The two groups and their **float64** sums always appear; the verbose flag adds the timings, the **float64** residual, and the high-precision sums and their difference. The difference comes out to about -1.43×10^{-13} — the exact optimum Knuth reported in his 1996 addendum, more than 5000 times better than his original 1976 ten-second solution.

⟨ Verify in high precision and report 14 ⟩ +≡

```
if *verbose {
    fmt.Printf("\nelapsed: %v\n", elapsed)
    fmt.Printf("best |T - (Asum+Bsum+k)| = %.3e (float64)\n", bestDiff)
    fmt.Printf("Asum=%.15f Bsum=%.15f k=%d\n\n", bestAsum, bestBsum, bestK)
}
fmt.Printf("Group1 (%d): %v\n sum = %.15f\n", len(group), group, sumG)
fmt.Printf("Group2 (%d): %v\n sum = %.15f\n", len(other), other, sumO)
if *verbose {
    fmt.Printf("\n[high-precision verification (math/big, 200-bit)]\n")
    fmt.Printf("sum1 = %s\n", bigG.Text('f', 40))
    fmt.Printf("sum2 = %s\n", bigO.Text('f', 40))
    fmt.Printf("sum1 - sum2 = %s\n", bigDiff.Text('e', 6))
}
```

16. Index.

⟨ Build the sorted table of A-subset sums 6 ⟩ Used in section 1
⟨ Define the *eval* helper 10 ⟩ Used in section 1
⟨ Define the *probe* helper 11 ⟩ Used in section 1
⟨ Parse the command-line flags 2 ⟩ Used in section 1
⟨ Reconstruct the winning partition 13 ⟩ Used in section 1
⟨ Scan B with a Gray code, probing each subset 12 ⟩ Used in section 1
⟨ Set the perfect squares aside 3 ⟩ Used in section 1
⟨ Set up the search state 9 ⟩ Used in section 1
⟨ Subroutines 7 ⟩ Used in section 1
⟨ Verify in high precision and report 14 ⟩ Used in section 1

Floyd's Partition Problem